

C++ Synchronized Buffered Ostream

ISO/IEC JTC1 SC22 WG21 P0053R1 - 2015-10-24

Lawrence Crowl, Lawrence@Crowl.org

Peter Sommerlad, Peter.Sommerlad@hsr.ch

Nicolai Josuttis

[Introduction](#)

[Solution](#)

[Design](#)

[Feature Test](#)

[Wording](#)

[17.6.1 Library contents \[contents\]](#)

[27.1 General \[input.output.general\]](#)

[27.3 Forward declarations \[iostream.forward\]](#)

[27.10 Synchronized output stream \[syncstream\]](#)

[27.10.1 Overview \[syncstream.overview\]](#)

[27.10.2 Class template basic_osyncstream \[syncstream.osyncstream\]](#)

[27.10.2.1 Constructor \[syncstream.osyncstream.ctor\]](#)

[27.10.2.2 Destructor \[syncstream.osyncstream.dtor\]](#)

[27.10.2.3 Member Functions \[syncstream.osyncstream.mfun\]](#)

[Implementation](#)

[Revisions](#)

[References](#)

Introduction

At present, stream output operations guarantee that they will not produce race conditions, but do not guarantee that the effect will be sensible. Some form of external synchronization is required. Unfortunately, without a standard mechanism for synchronizing, independently developed software will be unable to synchronize.

[N3535 C++ Stream Mutexes](#) proposed a standard mechanism for finding and sharing a mutex on streams. At the Spring 2013 standards meeting, the Concurrency Study Group requested a change away from a full mutex definition to a definition that also enabled buffering.

[N3678 C++ Stream Guards](#) proposed a standard mechanism for batching operations on a stream. That batching may be implemented as mutexes, as buffering, or some combination of both. It was the response to the Concurrency Study Group. A draft of that paper was reviewed in the Library Working Group, who found too many open issues on what was reasonably exposed to the 'buffering' part.

[N3665 Uninterleaved String Output Streaming](#) proposed making streaming of strings of length less than `BUFSIZ` appear uninterleaved in the output. It was a "minimal" functionality change to the existing standard to address the problem. The full Committee chose not to adopt that minimal solution.

[N3750 C++ Ostream Buffers](#) proposed an explicit buffering, at the direction of the general consensus in the July 2013 meeting of the Concurrency Study Group. In November 2014 a further updated version [N4187](#) was discussed in Library Evolution in Urbana and it was consensus to work with a library expert to get the naming and wording better suited to LWG expectations. Nico Josuttis volunteered to help the original authors. More information on the discussion is available at [LEWG wiki](#) and the corresponding [LEWG bug tracker entry \(20\)](#). This paper address issues raised with [N4187](#). This paper has a change of title to reflect a change in class name, but contains the same fundamental approach.

Solution

We propose a `basic_osyncstream`, that buffers output operations for a wrapped stream. The `basic_osyncstream` will atomically transfer the contents of an internal stream buffer to an ostream on destruction of the `basic_osyncstream`.

The transfer on destruction simplifies the code and ensures at least some output in the presence of an exception.

The intent is that the `basic_osyncstream` is an automatic-duration variable with a relatively small scope which constructs the text to appear uninterleaved. For example,

```
{
    ostream bout(cout);
    bout << "Hello, " << "World!" << endl;
}
```

Design

We follow typical stream conventions of `basic_` prefixes and typedefs.

The constructor for `ostream` takes a non-const reference to a `basic_ostream`. This non-const reference indicates that the destruction of the buffer may write to the stream associated with the argument.

The wording below permits implementation of `basic_ostream` with either a `stream_mutex` from [N3535](#) or with implementations suitable for [N3665](#), e.g. with Posix file locks [\[PSL\]](#)

Feature Test

No header called `<ostream>` exists; testing for this header's existence is thus sufficient for testing existence of the feature.

Wording

This wording is relative to N4527.

17.6.1 Library contents [contents]

Add a new entry to table 14:

<code><ostream></code>

27.1 General [input.output.general]

Add a new row to table 120.

27.10 Synchronized output streams <code><ostream></code>
--

27.3 Forward declarations [iostream.forward]

Add the following forward declarations to the synopsis of `<iosfwd>` in the namespace `std`.

```
template <class charT,
          class traits = char_traits<charT>,
          class Allocator = allocator<charT> >
    class basic_ostream;
typedef basic_ostream<char> ostream;
typedef basic_ostream<wchar_t> wostream;
```

27.10 Synchronized output stream [ostream]

Add a new section major section.

27.10.1 Overview [ostream.overview]

Add a new section.

The header `<ostream>` provides a mechanism to synchronize execution agents writing to the same stream. It defines a class template `basic_ostream` to buffer output and atomically transfer the buffered content into an `ostream`. The transfer occurs when `emit()` is called and when the `basic_ostream` object is destroyed.

Add a synopsis for header `<ostream>`.

```
template <class charT,
          class traits,
          class Allocator>
class basic_ostream;
```

27.10.2 Class template basic_ostream [ostream.ostream]

Add a new section.

```
template <class charT,
          class traits,
          class Allocator>
class basic_ostream
: public basic_ostream<charT,traits>
{
public:
    typedef charT char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits traits_type;
    typedef Allocator allocator_type;

    explicit basic_ostream(basic_ostream<charT,traits> &os);
    basic_ostream(basic_ostream&) = delete;
    ~basic_ostream();
    void emit();
    basic_ostream<charT,traits> & get() const;
    basic_ostream& operator=(basic_ostream&) = delete;
};
```

The class template basic_ostream supports buffering into an internal buffer and then atomically transferring the contents of the buffer to a basic_ostream.

[Example: Use a named variable within a block statement for streaming across multiple statements.

```
{
    ostream bout(cout);
    bout << "Hello, ";
    bout << "World!";
    bout << endl;
}
```

—end example]

[Example: Use a temporary object for streaming within a single statement.

```
ostream(cout) << "Hello, " << "World!" << endl;
```

—end example]

27.10.2.1 Constructor [ostream.ostream.ctor]

Add a new section.

```
explicit basic_ostream(basic_ostream<charT,traits> & os);
```

Effects: Stores the reference os, which will be the final destination of characters.

[Note: May construct a mutex. —end note]

Constructs a streambuf. Any flush on the streambuf will not act immediately on os, but will be deferred until immediately after the transfer of characters to os.

27.10.2.2 Destructor [ostream.ostream.dtor]

Add a new section.

```
~basic_ostream():
```

Effects: As if `emit()`.

[Note: May destroy a lock. —end note]

[Example: To protect against exceptions from within the destructor, use `emit()` to catch them.

```
{  
    ostream bout(cout);  
    bout << "Hello, " << "World!" << endl;  
    try {  
        bout.emit();  
    } catch ( ... ) {  
        // stuff  
    }  
}
```

—end example]

27.10.2.3 Member Functions [syncstream.ostream.mfun]

Add a new section.

```
void emit();
```

Effects: Transfers the contents of the internal `streambuf` to the stream specified in the constructor, so that they appear in the output stream as a contiguous sequence of characters, with respect to all other uses of `basic_ostream` on that stream. If and only if a flush was requested on the `streambuf`, the stream specified in the constructor will be flushed.

Synchronization: May or may not acquire a mutex while transferring characters.

```
basic_ostream<charT,traits>& get() const;
```

Returns: The reference to the wrapped `basic_ostream`.

[Example: Obtaining the wrapped `basic_ostream` with `get()` allows wrapping it again with a `basic_ostream`. For example,

```
{  
    ostream bout1(cout);  
    bout1 << "Hello, ";  
    {  
        ostream bout2(bout1.get());  
        bout2 << "Goodbye, " << "Planet!" << endl;  
    }  
    bout1 << "World!" << endl;  
}
```

produces the *uninterleaved* output

```
Goodbye, Planet!  
Hello, World!
```

—end example.]

Implementation

The `streambuf` class used by `basic_ostream` could be something along the lines of the following.

```
template <class charT, class traits, class Allocator>
```

```

class noteflush_streambuf
: public std::basic_stringbuf<charT,traits,Allocator>
{
    protected:
        virtual int sync();
};

```

The class template `noteflush_streambuf` behaves like a `basic_stringbuf` except that flushes of the stream using it, which incur in calling the `sync()` member function, are noted instead of being a no-op.

```
int sync() noexcept;
```

Effects: Notes the request of a flush within the object.

[*Note:* Any call to `sync()` ensures that a subsequent call to `emit()` in the owning `basic_ostream` will flush the stream being synchronized. —*end note*]

Returns: `0`.

Revisions

This paper revises [P0053R0 C++ Synchronized Buffered Ostream](#)

- List the new header in corresponding table.
- Provide type aliases in `<iosfwd>`.
- Removed copy constructor in favor of providing `get()`.
- Notify that move construction and assignment is deleted.
- Moved class `noteflush_streambuf` into an implementation note.
- Add a design subsection that states that a header test is a sufficient feature test.

This paper revises [N4187 C++ Ostream Buffers](#)

- Updated introduction with recent history.
- Rename `ostream_buffer` to `ostream` to reflect its appearance is more like a stream than like a buffer.
- Add an example of using `ostream` as a temporary object.
- Add an example of a `ostream` constructed with another `ostream`.
- Clarify the behavior of nested `ostream` executions.
- Clarify the behavior of exceptions occurring with the `ostream` destructor.
- Clarify the deferral of flush from the `ostream`'s `streambuf` to the final `basic_ostream`.
- Limit the number of references to `noteflush_stringbuf` in anticipation of the committee removing it from the specification.
- Rename `noteflush_stringbuf` to `noteflush_streambuf` to hide possible implementation details.
- Change the base class of `noteflush_streambuf` from `basic_stringbuf` to `basic_streambuf`.

[N4187](#) revised [N4069 C++ Ostream Buffers](#)

- Added note to `sync` as suggested by BSI via email.

[N4069](#) revised [N3978 C++ Ostream Buffers](#)

- Added a Design section.
- Clarify the reference capturing behavior of the `ostream_buffer` constructors.
- Added `noexcept` and `const` as appropriate to members.
- Added note on throwing wrapped streams.
- Change the `noteflush_stringbuf` public member variable `needsflush` to a public member query function `flushed`.

- Removed the public member function `noteflush_stringbuf::clear`.
- Minor synopsis formatting changes.
- Incorporated feedback from SG1 and Dietmar Kühl in specific in Rapperswil.

[N3978](#) revised [N3892 C++ Ostream Buffers](#)

- Flush the ostream if and only if the `ostream_buffer` was flushed.
- Add the `clear_for_reuse` function.
- Change the design from inheriting from `basic_ostream` to using a `noteflush_stringbuf`, which is a slightly modified `basic_stringbuf`. The modification is to note the flush rather than act upon it.

[N3892](#) revised [N3750 C++ Ostream Buffers](#)

- Change name to `basic_ostream_buffer` and add the usual typedefs.
- Change interface to inherit from `basic_ostringstream` rather than provide access to a member of that type.
- Add a Revisions section.

References

[PSL]

The Open Group Base Specifications Issue 6, IEEE Std 1003.1, 2004 Edition, functions, flockfile,
<http://pubs.opengroup.org/onlinepubs/009695399/functions/flockfile.html>